Author’s Workbench - Test Plan

1. Project Overview

This project is a React 19 + TypeScript + Vite web app. It builds to a single self-contained HTML file and deploys to GitHub Pages via GitHub Actions.

2. Test Objective

Every primary user flow (create/ edit/ delete/ reorder) based on user stories should be exercised at least once. Confirm domain calculations are correct at their boundaries, store operations never corrupt state, UI validation rules hold, and CI blocks any broken build or failing test from being deployed.

3. Test Environment

- OS: Windows/macOS
- Runtime: Node.js 24
- Test Framework: Vitest
- UI Testing: React Testing Library
- DOM Environment: jsdom
- Build Tool: Vite
- Language: TypeScript
- Linting: oxlint

4. Test Strategy

4.1 Unit Testing

Unit tests will verify individual functions in the domain layer.

Examples include:

- createProject()

- makeChapter()
- wordsWritten()
- makeCharacter()

The purpose is to verify that individual pieces of business logic return the correct results.

4.2 Component Testing

React components will be tested using Vitest and React Testing Library.

The tests will simulate user interactions such as:

- Clicking buttons
- Creating projects
- Cancelling modals
- Deleting projects

4.4 Compatibility

Manual check across latest Chrome, Firefox, Safari, Edge at desktop before release.

4.5 Integration Testing

Basic integration testing will verify that React components work correctly with the application store. This confirms that the UI and state-management layer work together correctly by driving multi-step flows.

(e.g. User creates project -> ProjectsPage -> WorkbenchProvider -> Project added to state -> Project displayed in UI)

4.6 Security

Dependency vulnerabilities will be checked using *npm audit*, and reproducible builds will be supported using *npm ci* with the committed lockfile.

4.7 Beta Testing

A beta version is deployed to the client before meetings.The client carries out the testing in real life situations. Product owner collects feedback from the client on Beta Testing and reports based on the feedback.

5. Manual Test

1. Run npm run dev and open the Vite local URL.
2. Verify the Projects page and empty state.
3. Open New project and verify the creation modal.
4. Enter a valid title, target word count and average chapter length.
5. Create the project and verify it appears.
6. Open the project and add at least two chapters.
7. Edit a chapter title and actual word count.
8. Move chapters and verify their order.
9. Delete a chapter and verify the remaining list.
10. Delete the project, verify confirmation and empty state.
11. Refresh the page and record current persistence behaviour.
12. Run *npm run test:run*, *npm run lint* and *npm run build*.

6. Test Deliverables

Test report including all capabilities and features in the set of tests, and a test coverage report.

7. Test Schedule

- Unit & component tests: run in CI on every push and pull request
- Integration tests: run in the same CI on every push, with manual testing
- Security tests: on every dependency change; manual review before releases
- Beta tests: automatic deployment on every merge to main for testing of the client; feedback collected timely